Nonlinear Dynamic System Identification Using Legendre Neural Network

Jagdish C. Patra and Cedric Bornand

***Abstract*— We propose a computationally efficient Legendre neural network (LeNN) for identification of nonlinear dynamic systems. Due to its single-layer architecture, the LeNN offers much less computational complexity than that of a multilayer perceptron (MLP). By taking several plant models of increasing complexity and with extensive simulations we have shown superior performance of the LeNN-based plant model in comparison to that of an MLP model in terms of estimated output, mean square error (MSE) and computational complexity, in presence of additive noise.**

I. INTRODUCTION

Accurate identification of a plant is crucial for its control and stability. In many practical applications, e.g., robotics, process control and autonomous systems, because of their nonlinear and dynamic characteristics, and absence of any *apriori* knowledge, to obtain an accurate model of the plant is a challenging task. Since early 1990, artificial neural networks (ANNs) have been the objective of numerous investigations because of their biological analogy and the record of successes obtained in such areas as machine learning and pattern recognition. The interest of the control community in neural networks increased after the appearance of the pioneering work of [1] and [2]. It is shown that ANN approaches to control and identification has high potential, especially when the plant information is not known apriri. Among the recently developed intelligent methods for control, ANNs have played a prevailing role in solving problems whose solutions would be hard to find with classical techniques [1]. Using 3-layer multilayer perceptrons (MLPs) they have shown that these networks are capable of modeling of complex dynamical systems effectively. Some of other reports on ANN application for system identification and control can be found in [3], [4], [5].

Besides MLP, many other ANN architectures have been used for pattern recognition, classification and system identification and control [6]. One of the major drawbacks of the MLP is that it is computationally intensive and needs large number of iterations for its training. In order to reduce the computational complexity, Pao et al. have introduced functional-link ANN (FLANN), and shown that it is capable of producing similar performance as that of MLP but with much less computational cost [7], [8]. A FLANN-based nonlinear dynamic system identification with satisfactory results has been reported [9]. It is shown that the performance of FLANN is similar to that of an MLP but with faster convergence and lesser computational complexity. A comprehensive survey on various applications of FLANN can be found in [10].

Because of their better representation capability, the Chebyshev polynomial based FLANN, called ChNN has been proposed for static function approximation [11] and pattern classification tasks [12]. It is shown that ChNN has the universal approximation capability. The ChNN has been extensively applied for nonlinear dynamic system identification [13], [14], [15], [16], digital communication channel equalization [17], [18], [19] and intelligent sensors [20], with quite satisfactory results. FLANN-based neurofuzzy network for nonlinear system control [21] and ChNN-based fuzzy recurrent network for modeling of servo-mechanism [22] have been proposed.

Yang and Tseng reported a novel Legendre polynomial-based linear ANN for static function approximation [23]. They have shown that this network has fast convergence, and provide high accuracy. Recently, a Legendre polynomial ANN, named LeNN, for channel equalization problem has been proposed and its superiority over MLP-based equalizer is shown [24], [25].

In this paper, we apply a novel LeNN for identification of nonlinear dynamic systems. By taking several nonlinear system models form [1] and with extensive computer simulation, we have shown that the LeNN-based model performs as good as MLP-based model but provides advantages in terms of computational complexity and faster convergence. The rest of this paper is as follows. In Section 2 we explain the basic problem of nonlinear system identification. In Section 3 we explain the basic architecture of the FLANN and LeNN and the learning algorithm. In Section 4 we provide details of the modeling examples and results of the ANN-based models. Finally, conclusions are drawn in the last Section.

II. SYSTEM MODELING

The system under study is first characterized by a mathematical model. Let us express the system model by an operator $\mathbf{P}$ from an input space $\mathscr{U}$ into an output space $\mathscr{Y}$. The objective is to categorize the class $\mathscr{P}$ to which $\mathbf{P}$ belongs. For a given class $\mathscr{P}$, $\mathbf{P} \in \mathscr{P}$, the identification problem is to determine a class $\hat{\mathscr{P}} \subset \mathscr{P}$ and $\hat{\mathbf{P}} \in \hat{\mathscr{P}}$ such that $\hat{\mathbf{P}}$ approximates $\mathbf{P}$ in some desired sense. In a static system, the spaces $\mathscr{U}$ and $\mathscr{Y}$ are subsets of $\mathscr{R}^n$ and $\mathscr{R}^m$, respectively. Whereas, in a dynamic system, they are assumed to be bounded Lebesgue integrable functions in the interval $[0,T]$ or $[0,\infty]$. However, in both cases, the operator $\mathbf{P}$ is defined implicitly by the specified input-output pairs [1].

J. C. Patra is with School of Computer Engineering, Nanyang Technological University, Singapore. E.mail: aspatra@ntu.edu.sg.

C. Bornand is with HEIG-VD, University of Applied Sciences, Yverdon-les-Bains, Switzerland. E.mail: Cedric.Bornand@heig-vd.ch.

978-1-4244-8126-2/10/$26.00 ©2010 IEEE

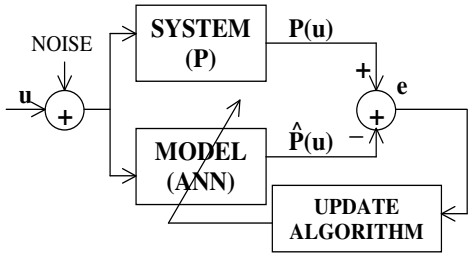

Fig. 1. Identification scheme of a dynamic system.

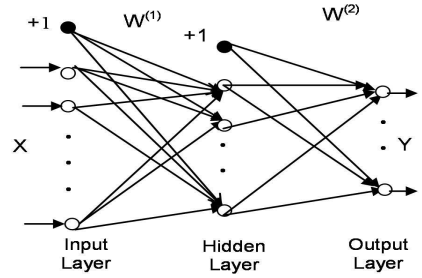

Fig. 2. Schematic diagram of an MLP.

A typical example of identification of static system is the problem of pattern recognition. By a decision function $\mathbf{P}$, compact input sets $U_i \subset \mathscr{R}^n$ are mapped into elements $y_i \in \mathscr{R}^m$ for $i = 1, 2, \cdots$ in the output space. The elements of U_i denote the pattern vectors corresponding to class y_i. Whereas, in a dynamic system, the input-output pairs of the time function $\{u(t), y(t)\}$, $t \in [0, T]$, implicitly define the operator $\mathbf{P}$ describing the dynamic plant. The main objective in both types of identification is to determine $\hat{\mathbf{P}}$ such that

$$\|y - \hat{y}\| = \|\mathbf{P}(u) - \hat{\mathbf{P}}(u)\| \quad < \varepsilon, \tag{1}$$

where $u \in \mathscr{U}$, ε is some desired small value > 0, and $\|\cdot\|$ is a defined norm on the output space. In Eq. (1), $\hat{\mathbf{P}}$ and $\mathbf{P}$ denote the output of the identified model and the plant, respectively. The error $e = y - \hat{y}$ is the difference between the observed plant output and the output generated by the model.

A schematic diagram of identification of a time-invariant, causal, discrete-time plant is shown in Fig. 1. The input and output of the plant are represented by u and $\mathbf{p}(u)$, respectively, where u is assumed to be an uniformly bounded function of time. The stability of the plant is assumed with a known parameterization but with unknown parameter values. The objective is to construct a suitable model generating an output $\hat{\mathbf{P}}(u)$ which approximates the plant output $\mathbf{P}(u)$. In the present study we considered single-input-single-output (SISO) plants, and four models are introduced below:

- Model 1:
$$y_p(k+1) = \sum_{i=0}^{n-1} \alpha_i y_p(k-i) + g[u(k), u(k-1), \cdots, u(k-m+1)]$$
- Model 2:
$$y_p(k+1) = f[y_p(k), y_p(k-1), \cdots, y_p(k-n+1)] + \sum_{i=0}^{m-1} \beta_i u(k-i)$$
- Model 3:
$$y_p(k+1) = f[y_p(k), y_p(k-1), \cdots, y_p(k-n+1)] + g[u(k), u(k-1), \cdots, u(k-m+1)]$$
- Model 4:
$$y_p(k+1) = f[y_p(k), y_p(k-1), \cdots, y_p(k-n+1); + u(k), u(k-1), \cdots, u(k-m+1)]$$

The input and the output of the SISO plant at the kth time instant are denoted by $u(k)$ and $y_p(k)$, respectively, and $m \leq n$. In this study two neural networks (MLP and LeNN) have been used to construct the nonlinear functions f and g so as to approximate such mappings over compact sets. It is assumed that the plant is bounded-input-bounded-output (BIBO) stable. In contrast to this, the stability of ANN model can not be assured. Therefore, in order to guarantee that the parameters of the ANN model converge, a series-parallel scheme is utilized. In this scheme, output of the plant, instead of ANN model, is fed back into the model during training of the ANN [1].

III. ANN Structures for System Identification

In this Section we briefly describe the ANN architectures for channel equalization based on MLP and LeNN.

A. The MLP

The MLP-based neural network, as shown in Fig. 2, is a multi-layered architecture in which each layer consists of certain number of nodes or processing units. A 2-layer MLP is denoted as $\{n_0, n_1\ n_2\}$, where, n_0, n_1 and n_2, denote the number of nodes (excluding the bias unit) in the input, hidden and output layers, respectively. The number of nodes in the input and the output layers are equal to the dimension of the input and output patterns, respectively. In a fully-connected MLP, each neuron of a lower layer is connected to all nodes of the upper layer through a set of connecting weights. Since no processing is done in the input layer, the output of its nodes is equal to the input pattern itself. In the nodes of other layers, processing is carried out as follows. The weighted sum of the node outputs of the lower layer is computed, then it is passed through a continuous nonlinear function (usually a $tanh(.)$ or *sigmoid* function).

During training phase, an input pattern is applied to the MLP, and the node outputs of all the layers are computed. The MLP output (output of the output layer nodes) is compared with the desired (or target) output to generate an error signal e_k. This error is used to update the weights of the MLP using a learning algorithm, e.g., back propagation (BP) or Levenberg-Marquardt (LM) algorithm [6]. Updating of connection weights is carried out until the MSE reaches a predefined small value.

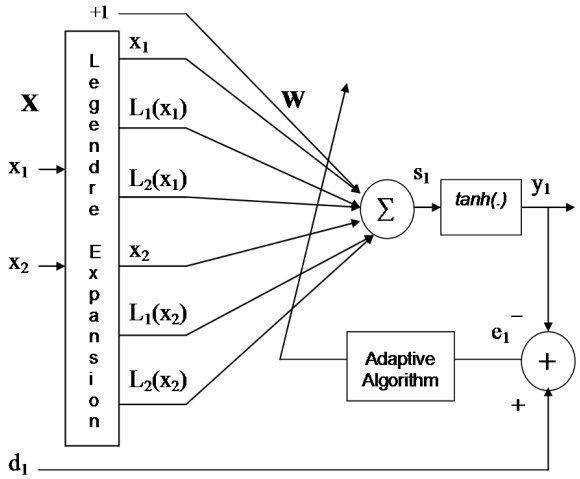

Fig. 3. Schematic diagram of an LeNN.

B. The Legendre ANN

The structure of an LeNN shown in Fig. 3 is a single-layer flat structure where the hidden layers are eliminated by transforming the input pattern to a higher dimensional space. In the projected higher dimensional space, the patterns are linearly separable. Due to the absence of hidden layer, LeNN provides computational advantage over the MLP. Using a set of Legendre orthogonal functions, the pattern enhancement is done in the LeNN. These functions take an element or the entire pattern as its argument.

The Legendre polynomials are denoted by $L_n(X)$, where n is the order and $-1 < x < 1$ is the argument of the polynomial. They constitute a set of orthogonal polynomials as solutions to the differential equation

$$\frac{d}{dx}\left[(1-x^2)\frac{dy}{dx}\right] + n(n+1)y = 0 \tag{2}$$

The zeroth and the first order Legendre polynomials are respectively given by, $L_0(x) = 1$ and $L_1(x) = x$. The higher order polynomials are

$$\begin{aligned} L_2(x) &= \frac{1}{2}(3x^2-1) \\ L_3(x) &= \frac{1}{2}(5x^3-3x) \\ L_4(x) &= \frac{1}{8}(35x^4-30x^2+3). \end{aligned} \tag{3}$$

The recursive formula to generate higher order Legendre polynomials is given by

$$L_{n+1}(x) = \frac{1}{n+1}\left[(2n+1)xL_n(x) - nL_{n-1}(x)\right] \tag{4}$$

As an example, a 2-dimensional input pattern $X = [x_1, x_2]^T$ is enhanced to a 7-dimensional pattern by Legendre functional expansion as $X^e = [1, L_1(x_1),\ L_2(x_1), L_3(x_1),\ L_1(x_2), L_2(x_2)\ L_3(x_2)]^T$.

C. Learning of LeNN

The learning process involves updating of the weights of LeNN in order to minimize a given cost function. Gradient descent algorithm is used for learning where the gradient of a cost function with respect to the weights is determined and the weights are incremented by a fraction of the negative gradient at each iteration. The well known BP algorithm is used to update the weights of LeNN. Consider a LeNN with a single output node. The goal of the learning algorithm is to minimize E_k, the cost function at kth instant, given by

$$E_k = \frac{1}{2}[y_k - d_k]^2 = \frac{1}{2}e_k^2 \tag{5}$$

where d_k is the desired output at the kth instant, and e_k $(= d_k - y_k)$ denotes the error term. In each iteration, an input pattern is applied, the output of the LeNN is computed and the error e_k is obtained. The error value is used in the BP algorithm to minimize the cost function until it reaches a pre-defined minimum value.

The weights are updated as follows:

$$W_{k+1} = W_k + \triangle W_k = W_k + \left(-\alpha\frac{\partial E_k}{\partial W_k}\right) \tag{6}$$

where α is a learning parameter which is set between 0 and 1. The gradient of the cost function (5) is given by

$$\frac{\partial E_k}{\partial W} = e_k\frac{\partial y_k}{\partial W} \tag{7}$$

The update rule for the weight w_{ji} is given by

$$w_{ji,k+1} = w_{ji,k} + \alpha e_{j,k}\frac{\partial y_{j,k}}{\partial w_{ji}} \tag{8}$$

If nonlinear $tanh(.)$ function is used at the output node, the update rule becomes

$$w_{ji,k+1} = w_{ji,k} + \alpha e_{j,k}\triangle w_{ji,k} \tag{9}$$

where $\triangle w_{ji,k} = (1-y_{j,k})^2\phi_i(X)$ and $\phi_i(X) = L_i(X)$. A momentum term β is added to the update rule to improve convergence, as given below:

$$w_{ji,k+1} = w_{ji,k} + \alpha\triangle w_{ji,k} + \beta\triangle w_{ji,k-1} \tag{10}$$

where the momentum factor is set between 0 and 1.

D. Computational Complexity

Let us consider an $L-$layer MLP with n_l number of nodes (excluding the threshold unit) in layer l, $l = 0, 1, \cdots, L$, where n_0 and n_L are the number of nodes in the input and output layer, respectively. An L-layer ANN architecture may be represented by $\{n_0 - n_1 - \cdots - n_{L-1} - n_L\}$. Three basic computations, *i.e.*, addition, multiplication, and computation of $tanh(\cdot)$ are involved for updating weights of the ANN. The computations in the network are due to the following requirements: (i) forward calculations to find the activation value of all the nodes of the entire network; (ii) back-error propagation for the calculation of square error derivatives; and (iii) updating the weights of entire network.

TABLE I

COMPARISON OF COMPUTATIONAL COMPLEXITY.

Number of	MLP	LeNN
Weights	$\mathscr{S}_1$	$n_1\mathscr{N}_0$
Additions	$3\mathscr{S}_2+3n_L-n_0n_1$	$2n_1\mathscr{N}_0+n_1$
Multi	$4\mathscr{S}_2+3\mathscr{S}_3-n_0n_1+2n_L$	$3n_1\mathscr{N}_0+n_0$
$tanh(\cdot)$	$\mathscr{S}_3$	n_1
$\mathscr{S}_1 = \sum_{l=0}^{L-1}(n_l+1)n_{l+1} \quad \mathscr{N}_0 = n_0+1$		
$\mathscr{S}_2 = \sum_{l=0}^{L-1} n_l n_{l+1} \quad \mathscr{S}_3 = \sum_{l=1}^{L} n_l$		

Total number of weights to be updated in one iteration in an MLP is given by $\Sigma_{l=0}^{L-1}(n_l+1)n_{l+1}$, whereas in case of a LeNN it is only $(n_0+1)n_1$. Since there is no hidden layer in the LeNN, computational requirement is drastically reduced compared to that of an MLP. A comparison of the computational requirements in one iteration of training using the BP algorithm, for two ANNs, is provided in Table I. In this Table, the number of multiplications and additions required to evaluate the Legendre polynomials are not included.

IV. SIMULATION STUDIES

Extensive simulation studies were carried out with several examples of nonlinear dynamic systems. We compared performance of the proposed LeNN with that of an MLP for the nonlinear dynamic systems reported by Narendra and Parthasarathy [1] and used the same MLP architecture {1-20-10-1} as used by them.

During the training phase, an uniformly distributed random signal over the interval [-1, 1] was applied to the plant and the ANN model. White Gaussian noise was added to the input of the plant. As it is usually done in adaptive algorithms, the learning parameter α and the momentum factor β in both ANNs were chosen after several trials to obtain best results. In a similar manner, the functional expansion of the LeNN was carried out. The adaptation continued for 50000 iterations during which the series-parallel identification scheme was used. Thereafter, adaption was discontinued. Using the adapted weights, the ANNs were applied for identification of the plants. During the test phase, effectiveness of the ANN models were analyzed by presenting a sinusoidal signal given by:

$$\begin{aligned} u(k) &= sin(2\pi k/250), \qquad 0 < \mathrm{k} \leq 250, \\ &= 0.8sin(2\pi k/250)+0.2sin(2\pi k/25), \mathrm{k} > 250. \end{aligned} \tag{11}$$

Note that this input was not seen by the ANN models during training as the ANNs were trained using only random signals. Performance comparison between the MLP and the LeNN was carried out in terms of output estimated by the ANN models, actual output of the plant and modeling error. A standard quantitative measure for performance evaluation is normalized mean square error (NMSE) and is defined as [26]:

$$\mathrm{NMSE} = \frac{1}{\sigma^2 \mathscr{N}_D}\sum_{k=1}^{\mathscr{N}_D}[y(k)-\hat{y}(k)]^2, \tag{12}$$

where $y(k)$ and $\hat{y}(k)$ represent the plant and the ANN model outputs at kth discrete time, respectively, and σ^2 denotes variance of the plant output sequence over the test duration $\mathscr{N}_D$. It may be noted that, in the results for all the examples provided below, the ANN model was trained with random signals. Where as, testing of the ANNs were carried out by applying a sinusoidal signal (11) to the plant and the model. The results are shown for 600 discrete samples, *i.e.,* $\mathscr{N}_D = 600$.

A. Example 1

We consider a system described by the difference equation of Model 1. The plant is assumed to be of second order and is described by the following difference equation:

$$y_p(k+1) = 0.3y_p(k)+0.6y_p(k-1)+g[u(k)], \tag{13}$$

where the nonlinear function g is unknown, but $\alpha_0 = 0.3$ and $\alpha_1 = 0.6$ are assumed to be known.

The unknown function g is given by: $g(u) = 0.6sin(\pi u)+0.3sin(3\pi u)+0.1sin(5\pi u)$. To identify the plant, a series-parallel model was considered which is governed by the following difference equation:

$$\hat{y}_p(k+1) = 0.3y_p(k)+0.6y_p(k-1)+\mathscr{N}[u(k)]. \tag{14}$$

The MLP used for the purpose of identification has a structure of $\{1-20-10-1\}$. For the LeNN, the input was expanded to 14 terms using Legendre polynomials. Both α and σ in (10) were chosen to be 0.5 in the two ANNs, and a white Gaussian noise of -10 dB was added to the input of the plant. The results of the identification with the sinusoidal signal (11) are shown in Fig. 4. It may be seen from this figure that the identification of the plant is satisfactory for both the ANNs. But, the estimation error in the LeNN is found to be less than that of the MLP. The NMSE for the MLP and the LeNN models are found to be -16.69 dB and -25.04 dB, respectively.

B. Example 2

We consider a plant described by the difference equation of Model 2:

$$y_p(k+1) = f[y_p(k),\, y_p(k-1)]+u(k). \tag{15}$$

It is known *apriori* that the output of the plant depends only on the past two values of the output and the input of the plant. The unknown function f is given by $f(y_1,y_2) = y_1y_2(y_1+2.5)(y_1-1.0)/(1.0+y_1^2+y_2^2)$. The series-parallel scheme used to identify the plant is described by:

$$\hat{y}_p(k+1) = \mathscr{N}[y_p(k),\, y_p(k-1)]+u(k). \tag{16}$$

A MLP of {2-20-10-1} structure was used. In the LeNN, the $2-$dimensional input vector was expanded by the Legendre polynomials to 12 terms. A Gaussian noise of -20

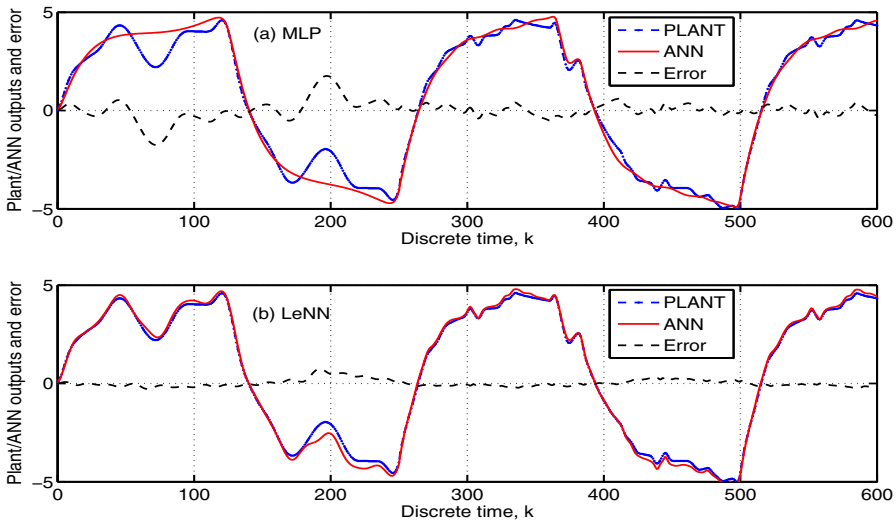

Fig. 4. Identification of nonlinear plant (Example 1) with test sinusoidal signal and additive noise of $-10dB$: (a) MLP and (b) LeNN.

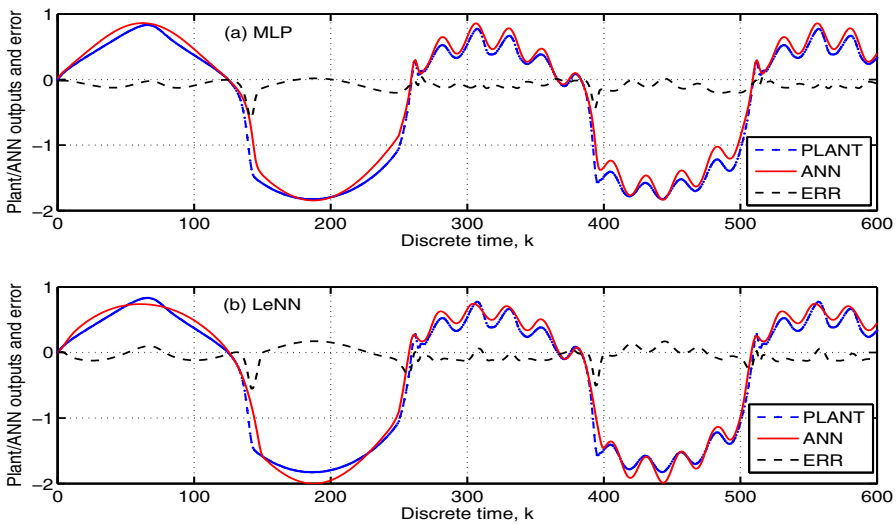

Fig. 5. Identification of nonlinear plant (Example 2) with test sinusoidal signal and additive noise of $-20dB$: (a) MLP and (b) LeNN.

dB was added to the input of the plant. For the MLP the values of α and β were set to 0.05 and 0.10, respectively. In the case of leNN, the values of α and β were chosen as 0.009 and 0.01, respectively. After completion of training, the sinusoidal signal (11) was applied to the plant and the ANN models. The results of the identification are shown in Fig. 5. The values of NMSE for the MLP and the LeNN are found to be -18.95 dB and -18.80 dB, respectively. It may be seen that the performance of both ANN models are similar and satisfactory.

C. Example 3

Here, the plant is of Model 3, and is described by the following difference equation:

$$y_p(k+1) = f[y_p(k)] + g[u(k)], \tag{17}$$

where the unknown functions f and g have are given by: $f(y) = \frac{y(y+0.3)}{1.0+y^2}$, and $g(u) = u(u+0.8)(u-0.5)$. The series parallel model for identification is given by

$$\hat{y}_p(k+1) = \mathcal{N}_1[y_p(k)] + \mathcal{N}_2[u(k)], \tag{18}$$

where $\mathcal{N}_1$ and $\mathcal{N}_2$ are the two ANNs used to approximate the two nonlinear functions f and g, respectively.

In the case of MLP, both $\mathcal{N}_1$ and $\mathcal{N}_2$ were represented by $\{1-20-10-1\}$ whereas, in the case of LeNN these were represented by $\{14-1\}$ structure. To improve the learning process, the output of the plant was scaled down by a scale factor (SF) before applying it to the ANN model. The SF

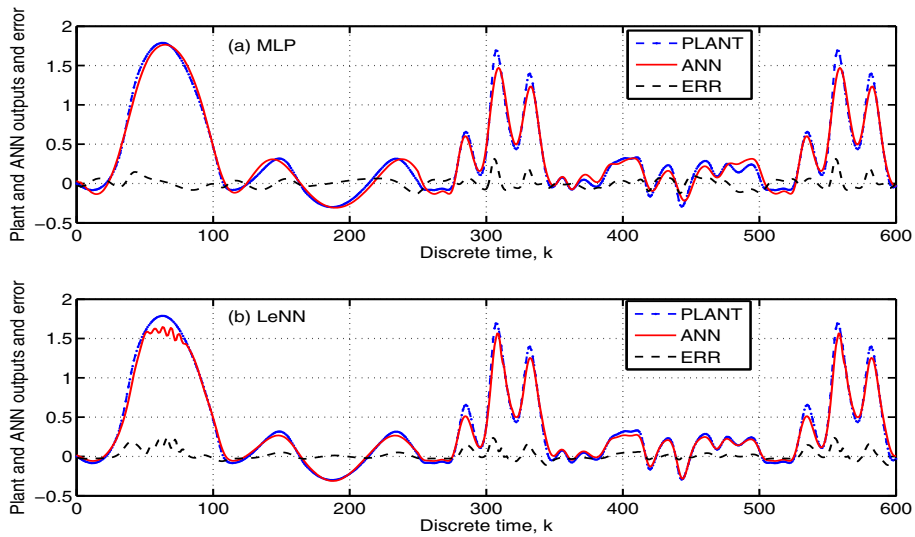

Fig. 6. Identification of nonlinear plant (Example 3) with test sinusoidal signal and additive noise of $-20dB$: (a) MLP and (b) LeNN.

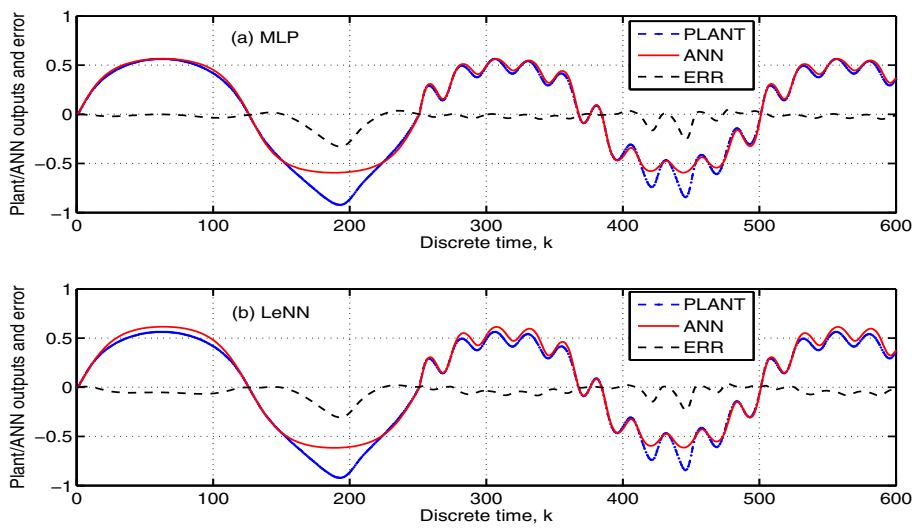

Fig. 7. Identification of nonlinear plant (Example 4) with test sinusoidal signal and additive noise of $-20dB$: (a) MLP and (b) LeNN.

was chosen as 2.0. The learning parameters α and β were chosen as 0.50 and 0.2, respectively, for the MLP. Whereas, in the case of LeNN, α and β were selected as 0.25 and 0.95, respectively. A Gaussian noise of -20 dB was added to the input of the plant. The results of the identification are depicted in Fig. 6. The NMSE values are found to be -19.45 dB and -19.87 dB for the MLP and LeNN models, respectively. It may be seen that the LeNN is able to estimate the plant response similar to that of the MLP.

D. Example 4

The plant model selected here is the most general of all the examples chosen. It belongs to the Model 4 and is described by the following difference equation:

$$y_p(k+1) = f[y_p(k), y_p(k-1), y_p(k-2), u(k), u(k-1)], \tag{19}$$

where the unknown nonlinear function f is given by $f[a_1, a_2, a_3, a_4, a_5] = (a_1 a_2 a_3 a_5 (a_3 - 1.0) + a_4)/(1.0 + a_2^2 + a_3^2)$. The series-parallel model for identification of the plant is given by

$$\hat{y}_p(k+1) = \mathcal{N}[y_p(k), y_p(k-1), y_p(k-2), u(k), u(k-1)]. \tag{20}$$

In the case of MLP and LeNN, $\mathcal{N}$ is represented by $\{5-20-10-1\}$ and $\{10-1\}$ structures, respectively. The inputs, y_p's and u's were expanded by using Legendre polynomials to 10 terms and used in the LeNN for identification of the plant. A Gaussian noise of -20 dB was added to input of the

plant and a SF of 0.7 was selected. The learning parameter α and the momentum factor β for the MLP model were chosen as 0.01 and 0.10, respectively. Where as, in case of LeNN, the α and β were selected as 0.02 and 0.10, respectively.

The outputs of the plant and the ANN models along with their corresponding errors are shown in Fig. 7. The NMSE for the MLP and LeNN models were found to be -15.56 dB and -15.53 dB, respectively. It may be observed that the performance of the LeNN is similar to that of the MLP model. From the above simulation results (Figs. 4-7), it may be seen that outputs of the ANN models agree with that of the plant models satisfactorily. The performance of the MLP and LeNN is found to be similar. The prime advantage of using LeNN is that its computational requirement is much less than that of MLP.

V. Conclusions

We have proposed a novel single-layer NN structure, named Legendre neural network, for identification of nonlinear dynamic systems. In the proposed LeNN, the input functional expansion is carried out using Legendre polynomials. In the four models of nonlinear dynamic systems considered in this study, the LeNN is found to be effective in identification of all the systems. The prime advantage of the proposed NN is its reduced computational complexity without compromising performance. Simulation results indicate that performance of the proposed network is as good as that of a MLP network in presence of additive noise. Because of computational advantage of LeNN, it may be used for other signal and image processing applications.

References

[1] K. S. Narendra and K. Parthasarathy, "Identification and control of dynamical systems using neural networks." *IEEE Trans. Neural Networks*, vol. 1, no. 1, pp. 4–27, March 1990.

[2] P. J. Antsaklis, "Neural networks for control systems." *IEEE Trans. Neural Networks*, vol. 1, no. 2, pp. 242–244, Jun. 1990.

[3] A. G. Parlos, K. T. Chong, and A. F. Atiya, "Application of recurrent multilayer perceptron in modeling of complex process dynamics," *IEEE Trans. Neural Networks*, vol. 5, pp. 255–266, Mar. 1994.

[4] N. Sadegh, "A perceptron network for functional identification and control of nonlinear systems." *IEEE Trans. Neural Networks*, vol. 4, no. 6, pp. 982–988, Nov. 1993.

[5] S. S. Ge, T. H. Lee, and C. H. Harris, *Adaptive neural network control of robotic manipulators*, World Scientific Series in Robotics and Intelligent Systems , vol. 19, 1998.

[6] S. Haykin, *Neural Networks, 2nd Ed.* Upper Saddle River, NJ: Prentice Hall, 1999.

[7] Y.-H. Pao, *Adaptive Pattern Recognition and Neural Networks*. Reading, MA: Addison-Wesley, 1989.

[8] Y.-H. Pao and S. M. Philips, "The functional link net and learning optimal control," *Neurocomputing*, vol. 9, pp. 149–164, 1995.

[9] J. C. Patra, R. N. Pal, B. N. Chatterji, and G. Panda, "Identification of nonlinear dynamic systems using functional linkartificial neural networks," *IEEE Transactions on Systems, Man and Cybernetics, Part B- Cybernetics*, vol. 29, no. 2, pp. 254–262, Apr. 1999.

[10] S. Dehuri and S.-B. Cho, "A comprehensive survey on functional link neural networks and an adaptive psobp learning for cflnn." *Neural Computing and Applications*, vol. DOI 10.1007/s00521-009-0288-5, July 2009.

[11] T. T. Lee and J. T. Teng, "The Chebyshev polynomial-based unified model neural network for function approximation," *IEEE Trans. Systems, Man, and Cybernetics, Part-B*, vol. 28, no. 6, pp. 925–935, Dec. 1998.

[12] A. Namatame and N. Uema, "Pattern calssification with Chebyshev neural networks," *Neural Networks*, vol. 3, pp. 23–31, March 1992.

[13] J. C. Patra and A. C. Kot, "Nonlinear dynamic system identification using Chebyshev functional link artificial neural network," *IEEE Trans. Systems, Man, and Cybernetics, Part B- Cybernetics*, vol. 32, pp. 505–511, Aug. 2002.

[14] H. Zhao and J. Zhang, "Nonlinear dynamic system identification using pipelined functional link artificial recurrent neural network," *Neurocomputing*, vol. 72, pp. 3046–3054, 2009.

[15] S. Purwar, I. N. Kar, and A. N. Jha, "On-line system identification of complex systems using chebyshev neural networks," *Applied Soft Computing*, vol. 7, pp. 364–372, 2007.

[16] ——, "Adaptive output feedback tracking control of robot manipulators using position measurements only," *Expert Systems with Applications*, vol. 34, pp. 2789–2798, 2008.

[17] J. C. Patra, W. B. Poh, N. S. Chaudhari, and A. Das, "Nonlinear chaneel equalization with QAM signal using Chebyshev artificial neural network," in *Proc. IEEE Int. Joint Conf. Neural Networks (IJCNN2005)*. Montreal, Canada, Aug. 1995, pp. 3214–3219.

[18] H. Zhao and J. Zhang, "Functional link neural network cascaded with Chebyshev orthogonal polynomial for nonlinear channel equalization," *Signal Processing*, vol. 88, pp. 1946–1957, 2008.

[19] W. D. Weng, C. S. Yang, and R. C. Lin, "A channel equalizer using reduced decision feedback Chebyshev functional link artificial neural networks," *Information Sciences*, vol. 177, pp. 2642–2654, 2007.

[20] J. C. Patra, M. Juhola, and P. K. Meher, "Intelligent sensors using computationally efficient Chebyshev neural networks." *IET Sci. Meas. Technol.*, vol. 2, no. 2, pp. 68–75, 2008.

[21] C.-H. Chen, C.-J. Lin, and C.-T. Lin, "A functional-link-based neurofuzzy network for nonlinear system control," *IEEE Trans. Fuzzy Systems*, vol. 16, no. 5, pp. 1362–1378, Oct. 2008.

[22] Y.-R. Huang, Y. Kang, M.-H. Chu, and Y.-P. Chang, "Modeling belt-servomechanism by chebyshev functional recurrent neuro-fuzzy network." *Journal of Advanced Mechanical Design, Systems, and Manufacturing*, vol. 2, no. 5, PAGES = 949-960, YEAR = 2008.

[23] S. S. Yang and C.-S. Tseng, "An orthogonal neural network for function approximation," *IEEE Trans. Systems, Man, and Cybernetics, Part-B*, vol. 26, no. 5, pp. 779–785, Oct. 1996.

[24] J. C. Patra, W. C. Chin, P. K. Meher, and G. Chakraborty, "Legendre-FLANN-based nonlinear channel equalization in wireless communication systems," in *Proc. IEEE Int. Conf. Systems, Man, Cybernetics (SMC2008)*. Singapore, Oct. 2008, pp. 1826–1831.

[25] J. C. Patra, P. K. Meher, and G. Chakraborty, "Nonlinear channel equalization for wireless communication systems using Legendre neural networks," *Signal Processing.*, vol. 89, pp. 2251–2262, 2009.

[26] N. A. Gershenfeld and A. S. Weigend, "The future of time series: Learning and understanding." in *Time Series Prediction: Forecasting the Future and Past, A. S.Weigend and N. A. Gershenfeld, Eds.* Reading, MA: Addison-Wesley, 1993, pp. 1–70.